

Project Title: Railway Reservation System

Group Number: TEAM 5

Technology Stack: C / C++

Team Profile

- **Ridhima Jain** | SRN: PES1UG24CS373
- **S S Adhithya Sriram** | SRN: PES1UG24CS393
- **Saathvik Shenoy Padubidri** | SRN: PES1UG24CS394
- **Samhitha S** | SRN: PES1UG24CS411

Proposed Project Description

The proposed software is a C/C++ railway reservation system designed to automate train scheduling, ticket booking, and queue management. To enhance UX without compromising the core backend logic, the system will feature a clean Text User Interface (TUI) built via **ncurses** (or a cleanly decoupled MERN/API wrapper)

The primary users are passengers and railway admins. Passengers need seamless interfaces to query schedules, analyze route efficiency, book tickets across travel classes, and process cancellations. Railway admins need elevated privileges to manage the backend inventory, configure train routes, and monitor daily operations.

At the conclusion of this project, users will perform the following functional features:

- Register as a new passenger and securely log into the system.
- Query available trains using a graph-based routing engine to find direct or connected multi-leg journeys.
- Filter and sort train options based on dynamic fares, travel time, and eco-routing metrics (e.g., fuel efficiency and carbon footprint per passenger).
- Book tickets and select preferred travel classes, resulting in a unique PNR generation.
- Automatically route excess bookings into a managed waitlist queue.
- Cancel existing tickets and automatically advance the waitlist queue to allocate freed seats.
- (Admin) Add, modify, or remove train schedules, node-based routes, and station halts.
- (Admin) View operational reports, including total bookings and revenue.

Plan of Work and Product Ownership: As per guidelines, UI design, DB interaction, and unit testing are classified as secondary responsibilities and do not constitute functional features.

The core C/C++ business logic is divided as follows to ensure equal workload weightage:

Short-Term Plan (Next 4 Weeks)

- **Sprint 1:** Finalize the Software Requirements Specification (SRS), set up Git for version control, and complete the initial user registration workflows.
- **Sprint 2:** Develop the core data structures (graphs/B-trees) for routes, the search algorithms, and environmental impact models.
- **Sprint 3:** Implement the transactional booking logic, PNR generation, and waitlist state machine.
- **Sprint 4:** UI and core logic, CI/CD pipeline, unit and system testing, and finalizing the validation report.

Team Functionality Split ("Who Should Do What") Each team member owns a distinct, fully realized functional feature from design through implementation.

- **Ridhima Jain - User Access & Profile Module:**
 - *Functionality:* The entire passenger and administrator onboarding process, secure credential verification, and Role-Based Access Control (RBAC).
 - *Secondary:* User profile UI flows and session management.
- **S S Adhithya Sriram - Inventory & Schedule Module (Admin):**
 - *Functionality:* Admin backend. Authorized users can define new train numbers, map multi-station routes, set departure times, and configure seat capacities across tiers.
 - *Secondary:* Admin dashboard UI and database mock schema generation.
- **Samhitha S - Search Engine, Fare & Eco-Optimization Module:**
 - *Functionality:* Core search and querying engine. Responsible for filtering train inventory based on user routing inputs and calculating dynamic fares based on distance, travel class, and passenger concessions.
 - *Secondary:* Core algorithm unit testing and performance benchmarking.
- **Saathvik Shenoy Padubidri - Booking & Queue Management Module:**
 - *Functionality:* Transactional logic. Decrements available seats, generates PNRs, pushes overflow into structured waitlist queues, and processes cancellations by automatically promoting waitlisted passengers.
 - *Secondary:* Payment state mock integration and waitlist UI views.